

1. Revise
 - @MappedSuperclass
 - @OneToMany (mappedBy,cascade,orphanRemoval)
 - @ManyToOne
 - @JoinColumn
 - cascading
2. More practice of entity associations

3. one-to-one
4. many-to-many
5. Value Types
6. Enter Spring (If time permits)

Today's Topics -

0. Answer this -

Consider below Code in Tester -

```
Category c1=new Category(....);
```

```
c1.addBlogPost(post1);
```

```
c1.addBlogPost(post2);
```

```
c1.addBlogPost(post3);
```

```
dao.saveCategory(c1);
```

In DAO method

```
session.persist(c1); commit;
```

What will happen

1. W/o Cascade

Ans - a parent rec will be inserted in categories table

2. With cascade=CascadeType.PERSIST | CascadeType.ALL

Ans - a parent rec will be inserted in categories table

+ 3 child recs will inserted in posts table

1. Establish uni dir associations (for more practice)

Comment *--->1 User(commenter)

Comment *--->1 BlogPost

2. Add validations

2.1 A blogger should not be allowed to comment on his/her own blog post

DAO layer soln

2.2 The SAME commenter be allowed to comment on the SAME POST again n again

3.

Solve

Post a new comment

i/p - comment text , rating , commenter id , post id

o/p status message

4. Display all comments for specified post

i/p - post title

o/p - list of comments displayed

Steps -

jpql="select c from Comment c where c.blogPost.title=:title"

5. Get category details by it's name
i/p category name

5.1 Display category details

5.2 Display post details belonging to that category.
Observation - LazyInitializationException

6. Problem root cause n solution for LazyInitializationException
Refer - "day9-data\day9_help\regarding lazyinit exception.txt"

7. Observe the select queries .
Why so many queries n so much of data n any way to reduce ?
i.e Along with Category , Post ,hibernate was lifting blogger(user) details also
Reason -Def fetch type for @ManyToOne -EAGER
So if your application doesn't need it all the time , can it be changed to LAZY ?

8. Establish one-one uni directional association between User n Address
User 1-->1 Address
How will you configure ?
Will you need cascading ?
FK column name - address_id
Questions -
User.java -
@OneToOne(cascade=CascadeType.ALL)
@JoinColumn(name="address_id")
private Address userAddress;

Address.java -NONE !
Where will you add , association related fields ?
Which annotations n it's attributes ???

8.1 Assign user address
i/p -- address details + user id
Steps in AddressDao
Observe the queries -

8.2 Lab work
Get user details along with it's address
i/p - user email
o/p -- user +address detail
Hint - jpql ="select u from User u left join fetch u.userAddress where u.email=:em"

8.3 List all users complete names from a specified city
i/p city name
o/p -- list of users (firstname n last name)
Hint - jpql ="select u from User u where u.userAddress.city=:city"

9. Add many-many uni dir association between

Tags *--->* Posts (w/o additional columns)

Tables -tags , posts -parent table

, tags_posts - child table (composite PK - tag_id, post_id : FK)

Entities - Tag , BlogPost

Annotations

Tag.java

Additional field -

@ManyToMany //mandatory

@JoinTable(name="my_tag_posts",joinColumns=@JoinColumn(name="tag_id")

,inverseJoinColumns=@JoinColumn(name="post_id"))

private Set<BlogPost> posts=new HashSet<>();

Tip - Instead of using List , use Set

(HashSet) in case of many-many , for reducing no of queries during removal.

Implement the contract between - hashCode n equals , in BlogPost class.

No need of additional entity here

Since there is no requirement of additional cols in the link table

Only cols required in tag_posts table - tag_id , post_id

Mandatory annotation - @ManyToMany

With this annotation , hibernate decides the name of the link table + column names.

How to customize link table names n composite PK column names - YES

@JoinTable

- name, joinColumns, inverseJoinColumns

9.1 Lab work(optional)

1. Create a new tag

i/p - tag name

2. Assign post to the tag

i/ps - tag name n post title

10. Till Now , we were looking at association(HAS-A) relationship between multiple entities.

Entities have a standalone existence(typically a separate table) , it's own PK.

It's life cycle(transient, persistent,detached,removed) is independent of any other entity

So this represents : weak association (aggregation)

Can there be a strong association between hibernate managed POJOs ? YES

11. Enter value types

Refer - "day9_help\readmes\Entity Vs Value Types.txt"

Refer - "day9_help\diagrams\JPA-Hibernate-POJO-Types.png"

User HAS-A Adhar Card : demo of entity n value type (composite)

User HAS-A PaymentCard (can have multiple credit/debit cards) : Collection of embeddable

User HAS-A Language Skills : string (multiple languages)

: collection of basic value type

Adhar Card : embeddable : @Embeddable
field : cardNo , date , loc

Lab work

Link adhaar card n languages to the user(blogger)

i/p card details , languages n user id

o/p - a mesg

NOTE -

JPA annotation

1. @Transient

Field / property(getter) level annotation

To tell JPA , to skip a field from persistence

(no column generation for the @Transient fields)

2. @Temporal

used for java.util.Date | Calendar | GregorianCalendar | java.sql.Date

use TemporalType.DATE | TIME | DATETIME